Dominic Lim *Soma Capital case study*

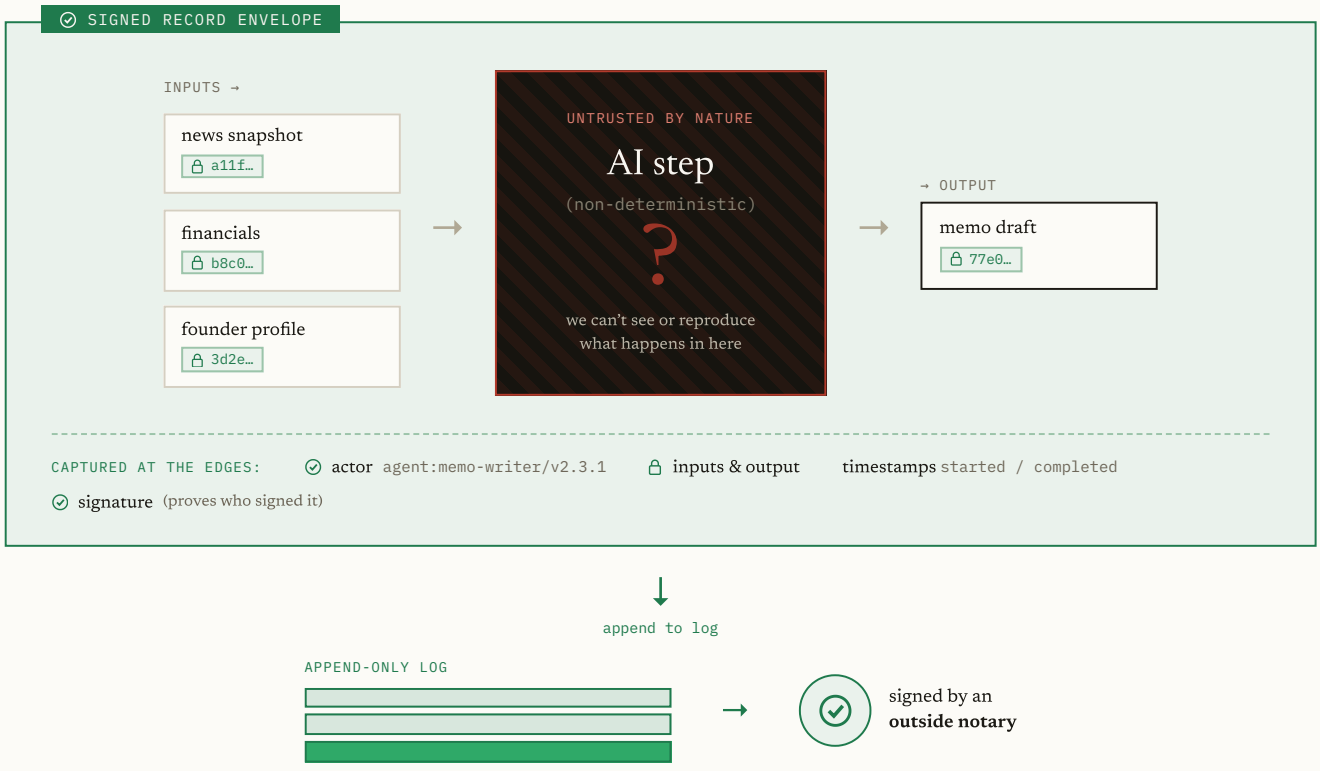
Approach

The design turns on one decision: you can't make an LLM deterministic, so don't try – make the *record* of what it did deterministic instead. Let the model be as unpredictable as it likes inside a step; the moment anything crosses the step's edge – the inputs it read, the output it produced, who ran it, when – capture it exactly, sign it, and append it to a log that can't be rewritten. The unpredictable part stays sealed inside; everything observable from outside is exact, attributed, and permanent.

"Can't be rewritten" is the load-bearing phrase. An ordinary database is append-only by *policy* – an admin can edit a row and cover their tracks. This design is append-only by *math*: records are chained by hash so changing any past record changes one fingerprint at the top, and that top fingerprint is signed by an outside party on a schedule. "Trust us, we didn't touch it" becomes "here's the proof, check it yourself."

At the brief's projection of 50,000 actions a day the system runs under one write per second, so throughput is never the constraint. The hard part is trust: the guarantee has to survive a compromised server, a dishonest insider, and an auditor with no reason to believe anything Soma says. The whole architecture is organized around surviving those three.

DIAGRAM 0 *The core idea – we can't control the inside, so we lock down the edges.*



"The inside is unpredictable. The edges are exact, signed, and permanent."

1 High-Level Architecture

The components fall out of the requirements

I didn't start by choosing components; I asked what the system must be able to *prove*, and the pieces followed.

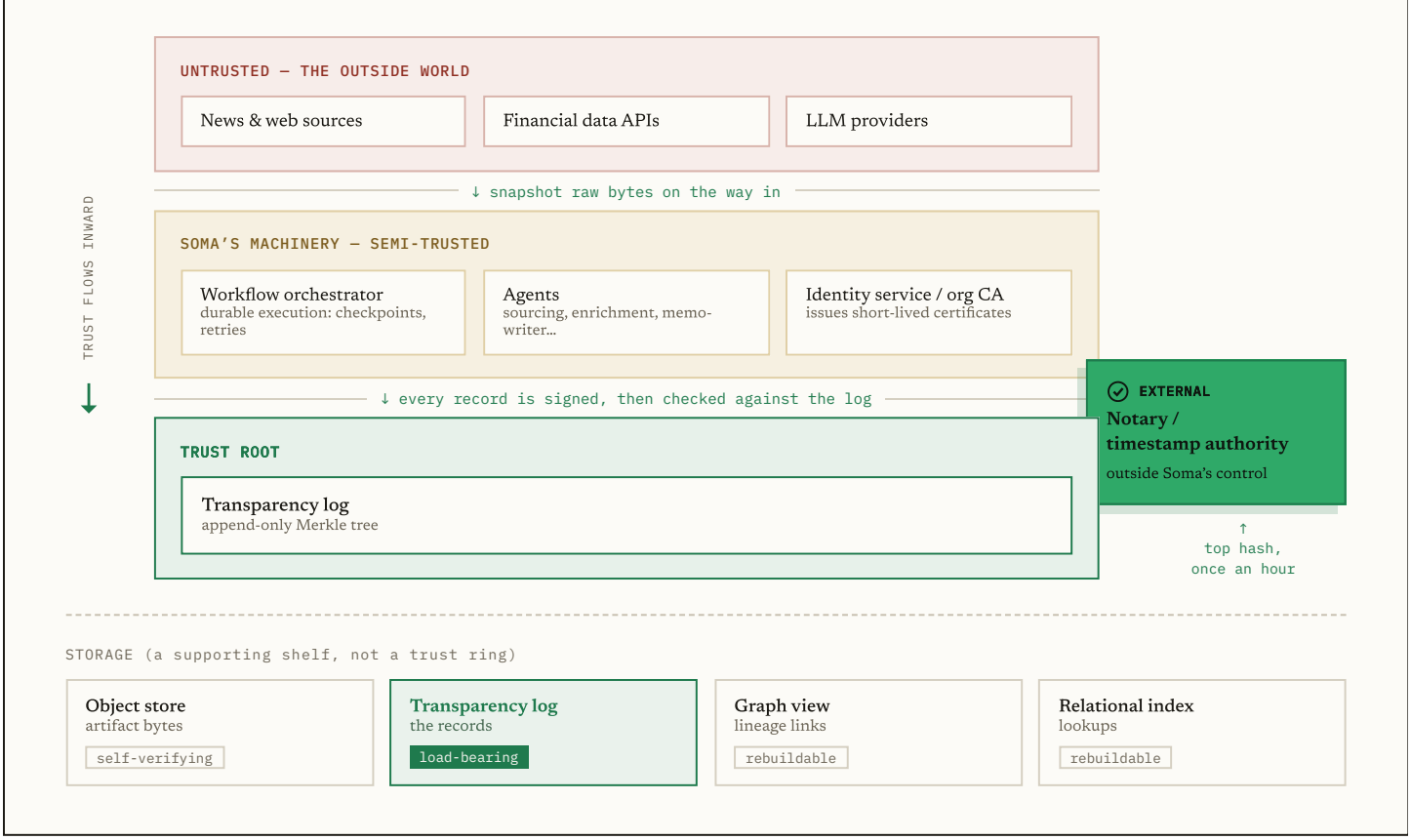
To answer *has this been modified*, an artifact can't be named by something a person can later point at different bytes — it's identified by its bytes, which forces a **content-addressed store** where the address *is* a fingerprint of the content. To answer *who made this, from what, when*, every step needs a small signed **record** of its actor, inputs, and output. For that record to be tamper-evident too, it can't sit in a database an admin can edit; it lives in an append-only **transparency log**, and because even the log's operator could rewrite it, an **external anchor** pins the log's state somewhere Soma can't reach.

The rest follow the same way: actors need identities and keys (the **identity service**); agents fail and retry across long runs without corrupting the record (a durable **workflow orchestrator**); and answering an auditor means walking records back from a decision into something checkable (the **audit layer**). Six components, each forced by a requirement rather than chosen for its own sake.

Trust boundaries

What you're allowed to trust is the whole game, since a guarantee is only as strong as its weakest dependency. The answer is three rings.

DIAGRAM 1 *Trust flows inward — only the small inner core is relied on.*

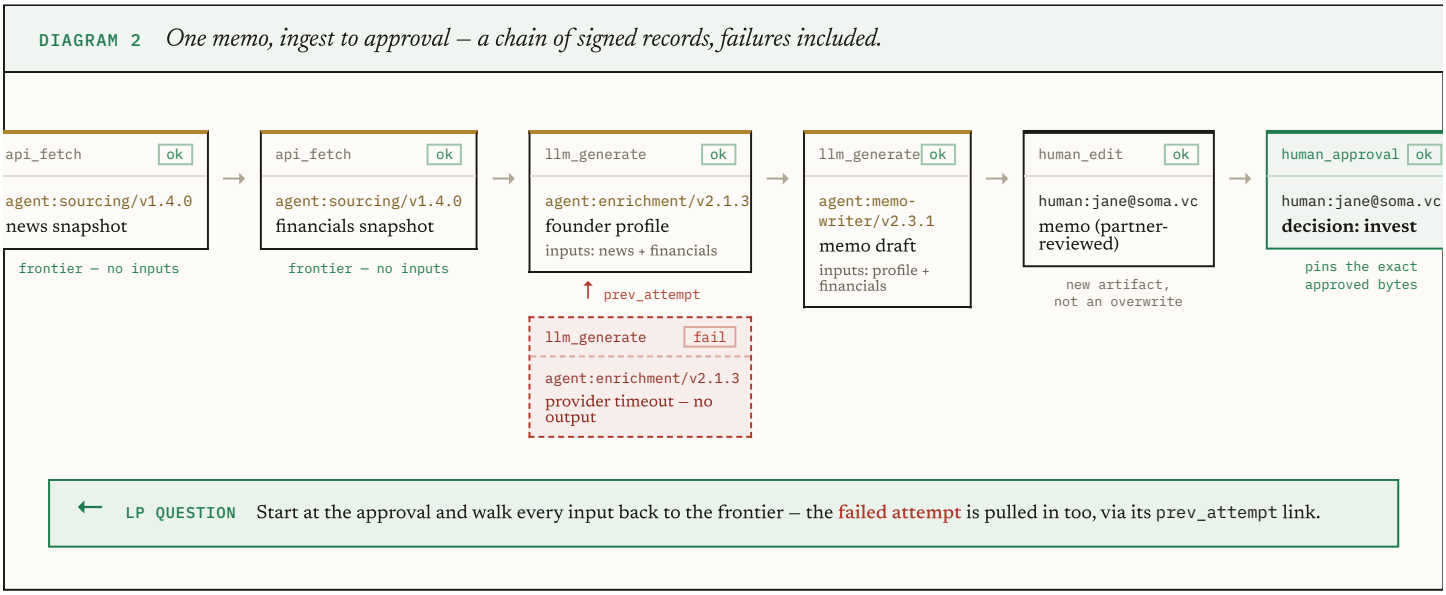


The **outer ring** is everything Soma doesn't control — news sites, financial APIs, LLM providers. It can be wrong, slow, or hostile, so nothing it says is taken as true; we snapshot exactly what it returns and treat that as a record of what was received, not a fact about the world. The **middle ring** is Soma's own machinery — orchestrator, agents, identity service. It does the work and holds signing keys, and the design's whole point is to *not* have to trust it: assume any single machine is compromised, and require that

a lie told here is caught by the inner ring. The **inner ring** is the trust root — the append-only log and the external notary — kept deliberately small because it's the one thing everyone relies on. Trust flows inward: the outer ring proves nothing, the middle ring's output is signed and then checked, and the inner ring is held honest by math and by an outsider.

Following one memo through the system

The clearest way to see the pieces work together is to follow one memo from raw sources to an approved decision, then watch it get audited. This is the exact path the prototype runs.



A sourcing agent fetches a news article and a financials response and saves the raw bytes of each — the first records, with no inputs, which makes them the *frontier* where outside data entered. An enrichment agent reads both and writes a founder profile; its first attempt times out, and that failure is written down as its own record, with the successful retry pointing back at it. A memo agent drafts from the profile, a partner edits the draft (creating a new document rather than overwriting it), and the approval pins the exact bytes approved. When the LP later asks the headline question, the system starts at that approval and walks the input links back to the two frontier snapshots — artifacts from the content store, each step a signed record, the whole chain reconstructed on demand.

Durable workflows, and the boundary the brief asks about

Reliable execution is a solved, purchasable problem: a durable engine (Temporal and kin) checkpoints progress so a crash resumes, and uses idempotency keys so a retried write happens once. I don't rebuild that. What I own is where reliability and provenance collide — retries.

The two kinds of step are opposites. A **deterministic** step (a DB write, an API call) can be re-run safely; the idempotency key absorbs the duplicate. A **non-deterministic** step (an LLM call) can't be re-run to check it, because a second run gives a different answer — so its output is recorded once and reused, never re-executed. The consequence is the decision that matters: every attempt, including the failed one, becomes its own permanent record, and the retry links back to the failure. A history that drops that failed attempt fails verification, so the boundary is enforced by math, not policy.

Identity, and the trust model

A signature only means something if you know whose key produced it and that the key couldn't have been quietly stolen months ago. The obvious approach — long-lived keys plus a revocation list — is the wrong one, because revocation is where these systems fail in practice: a stolen key stays valid until a human notices. So an org CA issues **short-lived certificates**, good for hours, to each agent instance. Rotation takes care of itself — an agent requests a fresh cert as its current one nears expiry — and there's no long-lived secret to steal. The cost, stated honestly, is an always-on issuer: if it's down, agents can't sign. At the rate agents work, keeping an issuer up is far easier than keeping a revocation list correct and fast; for an identity that had to sign offline, the trade would flip.

Two pieces finish the model. An agent never acts on its own authority — its certificate must carry a delegation chain terminating at a real human, or verification rejects it. And holding a valid key isn't enough to sign *as* someone else: the identity claimed in a record must match the certificate the CA issued. Humans authenticate through Soma's existing SSO with hardware-backed keys. The model in a sentence: trust the org CA for who is who, an outside notary for when history existed, and nothing else — including Soma's own servers.

Storage, and the five questions

Four storage layers, and only one has to be trusted. Artifact bytes sit in ordinary object storage (self-verifying, since the address is the hash). The records sit in the append-only log, which with the external anchor is the only load-bearing layer. A graph view makes the audit walk fast and a relational index answers ordinary lookups, but both are *derived*: an auditor who trusted none of Soma's databases could throw them away, rebuild from the log, and every proof would still hold. With that in place, the brief's first requirement — five questions each artifact must answer — resolves plainly:

TABLE 1 *The five provenance questions, answered in their words.*

"Which agent or human created this?"	Every step names its actor — a person or a specific agent version — signed in, so nobody can put someone else's name on their work.
"What inputs or sources were used?"	The record lists every input, each pointing at the exact earlier artifact. Follow the pointers back for the full family tree.
"What version of the agent produced it?"	The version is in the actor's name; for AI steps the record also stores the model, prompt, and settings.
"When was it created?"	Start and completion times are in the record — claimed by the actor, with the notary proving when a whole batch of history existed.
"Has it been modified?"	An artifact's name *is* the fingerprint of its bytes — change it and the name changes. Re-checked on every read.

2 Deep Dive: the record, the DAG, and the log

Depth on one component. I chose this core, because it's where "who made this, from what, and has it changed" is actually answered — and where a plausible-looking design can be quietly, dangerously wrong. Three things do the work: the record that captures a step, the hash-links that turn records into lineage, and the log that makes the history complete and unrewritable.

What a record captures, and why

Start from the five questions and let them force the contents. *Who* forces an actor identity and a signature over the whole record. *From what* forces a list of the exact upstream artifacts, each by content fingerprint. *Which version* forces the agent's version into its name, plus, for an AI step, the model and its settings. *When* forces start and completion times. And *has it been modified* comes free from referencing the output by fingerprint — any change breaks every record that pointed at it.

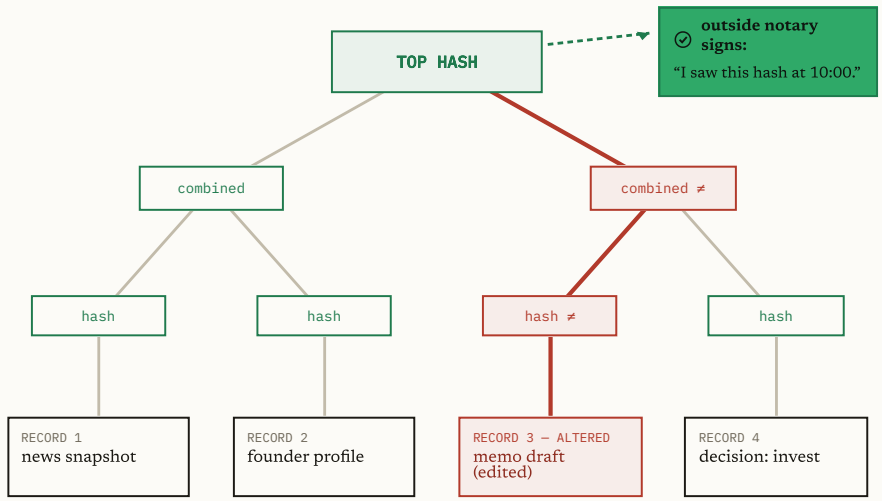
Two choices earn their keep: referencing inputs and outputs by fingerprint rather than embedding bytes (so a record stays about a kilobyte, and deleting an artifact for privacy never disturbs the records that point at it), and making a failed attempt a first-class record the retry links back to (so hiding a failure is structurally impossible, not merely discouraged). One field exists purely for the AI problem: we can't record *why* a model produced an output, but we record everything that went *in* — model and version, exact prompt, settings, the provider's request ID. The step is fully attributable even though it can never be reproduced. That distinction — attributable but not reproducible — is the honest center of putting AI in an audit trail.

Hash-links are the lineage, and the log makes it complete

Each record names its inputs by fingerprint, and each input was an earlier record's output that named *its* inputs by fingerprint, back to the frontier. So the records form a graph whose edges are fingerprints sealed inside signed records — faking an ancestor would need either two files with the same fingerprint (infeasible by design) or a forged signed record (which can't be slipped into the log after the fact). Lineage and tamper-resistance become one mechanism.

But a signature only stops a record from being *changed* — not deleted, and not shadowed by a second, fully-signed, invented history. So the records go into a structure where no one, not even the operator, can delete one or swap in a fake without it showing: hash every record, hash them in pairs, and again, up to a single top hash that fingerprints the entire history at once. This is a Merkle tree, the same construction that keeps the world's website certificates honest under Certificate Transparency.

DIAGRAM 4 *Change one record and the single number at the top changes — so no one can rewrite the past quietly.*



ONE CHANGE RIPPLES ALL THE WAY UP

Edit **Record 3** and its hash changes. That changes the combined hash above it, which changes the **top hash** — the one number that stands for the entire history.

But the notary already signed the *old* top hash. So the rewritten history no longer matches what an outsider vouched for — the edit is caught, not hidden.

Prove one record is in the history
Show the record plus a handful of neighbouring hashes (~25, even for millions of records) — not the whole log.

Prove today still contains all of yesterday
Yesterday's smaller tree nests inside today's — nothing old was removed or changed, only new records added on the end.

Two proofs fall out for free — exactly the two an auditor needs. You can prove one record is in the history with just the record plus about 25 sibling hashes, not the whole log; and you can prove today's history still contains all of yesterday's, unchanged, with new records only appended. The one remaining hole is that all of this assumes someone kept yesterday's top hash — so once an hour the current top hash goes to an outside timestamping notary that signs "I saw this hash at this time." From then on it's pinned where Soma can't reach, and rewriting even one old record fails to produce a valid proof back to it. Rewriting history stops being forbidden by policy and becomes detectable by arithmetic.

Answering the LP with a proof that needs no trust in Soma

The audit layer takes the approval artifact, walks the input links back to the frontier (pulling in failed attempts), and packages a proof bundle: every record with its inclusion proof, the current and last-anchored top hashes with a consistency proof between them, every certificate, the notary's receipt, and optionally the bytes. What makes it a *proof* rather than a report is that a standalone verifier checks all of it while trusting exactly two public keys — the org CA and the notary — sharing no database or code with the system that produced it. A malformed bundle returns a failure instead of crashing, so a crash can't be mistaken for a pass. The four ways someone would actually cheat all fail against it:

TABLE 3 *Four ways to cheat — and how each one is caught.*

THE CHEAT	HOW IT'S CAUGHT
Edit an approved memo after the fact	The disclosed bytes no longer match the artifact's fingerprint.
Forge a record under a same-named lookalike CA	The certificate doesn't chain to the real CA, and the altered record isn't in the log.
Hide the failed attempt	The retry's "previous attempt" link points at a record that's missing.
Rebuild the whole log with one record swapped	The rebuilt history can't produce a consistency proof back to the notary's hash.

Building it taught the real lesson. An adversarial review found two bugs the original tests missed, both failures of *attribution* — the exact property the system exists to guarantee — and both invisible until someone deliberately lies. Expired certificates could pass on a broken timestamp (in JavaScript every comparison against an invalid date is false, so garbage read as "in range"); fixed by failing closed. And identity was signed but not *bound* — the verifier never checked that the claimed identity matched the certificate's subject, so any key holder could attribute work to someone else; fixed by requiring an exact match. The dangerous bugs aren't in what a verifier checks, but in what it silently forgets to — which is why the most valuable tests are the ones that lie to it.

3 Tradeoffs

TABLE 4 *What the design optimizes for — and what it gives up, on purpose.*

OPTIMIZED FOR	GIVEN UP, ON PURPOSE
Verifiability by an outside party who doesn't trust Soma	Throughput (one write per second — nothing to optimize)
Provable completeness, via the Merkle log	Storage cleverness (small volumes; boring storage is easier to trust)
An insider with database access still can't rewrite history undetected	Sub-minute audit latency (indexes are rebuilt from the log, not kept always-correct)
Proofs that travel to the auditor, verifiable off Soma's systems	

Every choice traces back to one thing: verifiability by a party who doesn't trust Soma. What I gave up, I gave up deliberately — throughput was never in question at one write per second, storage stayed boring on purpose (boring is easier to trust), and audit queries may take the minute the brief allows, which lets indexes be rebuilt from the log rather than maintained as exotic always-correct ones.

The forks worth naming: a **blockchain** would remove the trusted log operator entirely, but a permissionless consensus system is wildly out of proportion for an internal tool at one write per second whose rewrite window is already an hour — kept as an escape hatch (anchor the top hashes into a public chain) if that trust requirement ever arrives. A **separate lineage database plus audit log** would be simpler but splits lineage and integrity into two systems that can drift; making the signed record stream itself be the lineage removes the seam. **Re-running AI steps to verify them** stays available for deterministic steps but can't work for LLMs today, so the design commits to attributable-but-not-reproducible with a clean upgrade to provider-signed inference when it exists.

The limits, stated plainly: it proves lineage, not truth — a hallucination is recorded with perfect integrity, which is why decision-grade artifacts require a human approval. A live compromised endpoint can sign real lies with a valid key; short certificate windows and the anchored log contain that but don't eliminate it. Outside data is trust-on-first-fetch, fingerprinted at the earliest moment. And record timestamps are self-claimed, with the anchor proving only when a batch existed — per-record

countersigned timestamps are the fix, and the interface is ready for them. At 10x to 100x nothing in the trust core changes shape; the log shards under a shared super-root, anchoring batches, cold bytes tier to archival while fingerprints stay hot, and the derived views scale freely. What you could prove to the LP on day one is what you can still prove on the ten-thousandth.